MVC, Decorators e FK no SQLAlchemy

1. Padrão MVC (Model – View – Controller)

MVC é uma forma de organizar o código separando **responsabilidades**. A ideia central é que cada parte do sistema deve fazer **uma só coisa** e não saber dos detalhes das outras.

```
main.py
  │
  ├── Controller  → "o que fazer" (lógica, regras, banco)
  ├── Model       → "como os dados são" (tabelas, colunas, ti
pos)
  └── View        → "como mostrar" (print, formatação)
```

Model

Define a estrutura dos dados. No projeto, cada Model é uma classe Python que o SQLAlchemy mapeia para uma tabela SQL.

```
class Usuario(Base):
    __tablename__ = "usuarios"
    id    = Column(Integer, primary_key=True)
    nome  = Column(String(100))
    email = Column(String(150), unique=True)
```

> **Regra:** o Model nunca imprime nada e nunca decide o que fazer com os dados — só define como eles são.

Controller

Contém a lógica da aplicação: abre sessão, faz a operação, trata erros, fecha sessão.

```
def inserir_usuario(nome, email):
    s = Usuario.session()
    try:
        u = Usuario(nome=nome, email=email)
        s.add(u)
        s.commit()
        s.refresh(u)
        return u
    except Exception as e:
        s.rollback()
        raise e
    finally:
        s.close()
```

> **Regra:** o Controller nunca imprime nada — só retorna dados ou lança exceções.

View

Recebe objetos prontos e decide como exibi-los. Não sabe nada de banco de dados.

```
def exibir_usuario_criado(usuario):
    print(f"[+] {usuario.nome} — {usuario.email}")
```

> **Regra:** a View nunca abre sessão nem calcula nada além de formatação.

Por que separar assim?

Camada	Se mudar...	Afeta
Model	Adicionar coluna	Só o Model e o Controller

Camada	Se mudar...	Afeta
Controller	Trocar SQLite por PostgreSQL	Só o Controller
View	Mudar de terminal para HTML	Só a View

Sem MVC, uma mudança no banco pode quebrar a tela, e uma mudança visual pode introduzir um bug de dados.

---

2. Decorators em Python

Um decorator é uma função que **envolve outra função** para adicionar comportamento sem alterar o código dela.

```
@decorator
def minha_funcao():
    ...
```

É equivalente a:

```
minha_funcao = decorator(minha_funcao)
```

@staticmethod

Agrupa uma função dentro de uma classe **por organização**, sem precisar de `self` (instância) nem `cls` (classe).

```
class Usuario(Base):
    @staticmethod
    def banco():
        return create_engine("sqlite:///usuarios.db")
```

Quando usar: a função não precisa acessar nenhum dado da instância nem da classe — é só um utilitário que faz sentido ficar dentro dessa classe.

@classmethod

Recebe a **própria classe** como primeiro argumento (`cls`). Usado para operações que dizem respeito à classe inteira, não a uma instância específica.

```
class Usuario(Base):
    @classmethod
    def criar_tabela(cls):
        Base.metadata.create_all(cls.banco())
```

A diferença para `@staticmethod`:

```
Usuario.criar_tabela()   # @classmethod — cls = Usuario
Usuario.banco()          # @staticmethod — nenhum argumento i
mplícito
```

@property e @setter

Transforma um método em um **atributo de leitura** que executa código quando é acessado ou modificado. Isso é a base do encapsulamento.

```
class Produto(Base):
    _preco = Column("preco", Float)

    @property
    def preco(self):           # leitura: produto.preco
        return self._preco

    @preco.setter
    def preco(self, valor):    # escrita: produto.preco = 50.
0
        if valor < 0:
            raise ValueError("Preço não pode ser negativo.")
        self._preco = valor
```

Sem `@property`, seria necessário chamar `produto.get_preco()` e `produto.set_preco(50)`, o que é mais verboso e quebra a interface natural do objeto.

> **Por que o atributo de banco se chama `_preco` e a coluna se chama `"preco"`?**

> O SQLAlchemy usa o nome do atributo Python como nome da coluna por padrão. Como o atributo privado se chama `_preco`, precisamos passar o nome real da coluna explicitamente:

> `python

> _preco = Column("preco", Float)

> # ↑ nome da coluna no banco

> `

---

3. Foreign Key (FK) e Relacionamentos no SQLAlchemy

O que é uma Foreign Key?

É uma coluna que **referencia a chave primária de outra tabela**. Garante que o banco não aceite um valor inválido nessa coluna — integridade referencial.

```
tabela clientes          tabela pedidos
_________________        ____________________________
id | nome                id | descricao   | cliente_id
___|______               ___|____________|____________
1  | Ana                 1  | Notebook    | 1   ← válido
2  | Bruno               2  | Mouse       | 99  ← ERRO: cliente
99 não existe
```

No SQLAlchemy:

```python
class Pedido(Base):
    cliente_id = Column(Integer, ForeignKey("clientes.id"), n
```

```
ullable=False)
    #                                ↑ "nome_da_tabela.coluna"
```

relationship()

Enquanto `ForeignKey` cria a coluna no banco, `relationship()` cria o **atalho Python** para navegar entre objetos sem escrever JOINs manualmente.

```
class Cliente(Base):
    pedidos = relationship("Pedido", back_populates="client
e")

class Pedido(Base):
    cliente_id = Column(Integer, ForeignKey("clientes.id"))
    cliente    = relationship("Cliente", back_populates="pedi
dos")
```

Com isso:

```
cliente.pedidos   # lista todos os pedidos desse cliente
pedido.cliente    # retorna o objeto Cliente desse pedido
```

O `back_populates` informa ao SQLAlchemy que os dois lados se referenciam mutuamente — ele mantém os dois sincronizados automaticamente.

Tipos de relacionamento

1:N (um para muitos) — ex4, ex10

Um cliente tem vários pedidos. A FK fica no lado "muitos" (pedidos).

```
# ex4 — Cliente tem vários Pedidos
class Pedido(Base):
    cliente_id = Column(Integer, ForeignKey("clientes.id"))
```

```
clientes (1) ──────< pedidos (N)
```

1:N com dados extras — ex5

`ItemPedido` é uma tabela entre `Pedido` e `Produto` que guarda dados da relação (quantidade, preço no momento da compra).

```
pedidos (1) ──────< itens_pedido (N) >────── produtos (1)
```

```
class ItemPedido(Base):
    pedido_id      = Column(Integer, ForeignKey("pedidos.i
d"))
    produto_id     = Column(Integer, ForeignKey("produtos.i
d"))
    quantidade     = Column(Integer)
    preco_unitario = Column(Float)   # preço "congelado" no m
omento da compra
```

N:N (muitos para muitos) — ex6

Aluno pode ter vários cursos; curso pode ter vários alunos. Precisa de uma **tabela de junção** para representar isso no banco relacional.

```
alunos (N) ──────< aluno_curso >────── cursos (N)
```

Tabela de junção simples (sem atributos extras) — usa `Table`:

```
aluno_curso = Table(
    "aluno_curso", Base.metadata,
    Column("aluno_id", Integer, ForeignKey("alunos.id"), prim
ary_key=True),
    Column("curso_id", Integer, ForeignKey("cursos.id"), prim
ary_key=True),
)
```

O `secondary` no `relationship` aponta para essa tabela, e o SQLAlchemy gerencia os INSERTs nela automaticamente:

```
aluno.cursos.append(curso)  # insere uma linha em aluno_curso
```

Herança com tabelas separadas — ex7

Cada subclasse tem sua própria tabela ligada à tabela pai pelo mesmo `id`.

```
pagamentos (id, valor, tipo)
    │
    ├── pagamentos_cartao (id FK, numero_cartao, bandeira)
    └── pagamentos_pix    (id FK, chave_pix)
```

A coluna `tipo` é o **discriminador** — o SQLAlchemy a usa para saber qual classe Python instanciar ao ler do banco.

```
__mapper_args__ = {
    "polymorphic_on":       tipo,       # coluna que diferenc
ia
    "polymorphic_identity": "cartao",   # valor gravado para
esta subclasse
}
```

cascade="all, delete-orphan" — ex10

Quando um `Usuario` é deletado, todas as suas `Tarefa`s devem ser deletadas junto. Sem o cascade, o banco lançaria erro de integridade referencial (FK apontando para usuário inexistente).

```
class Usuario(Base):
    tarefas = relationship("Tarefa", back_populates="usuari
o",
                           cascade="all, delete-orphan")
```

flush() vs commit() — ex5

Operação	O que faz
`s.flush()`	Envia o SQL ao banco, mas **não finaliza** a transação. O id fica disponível, mas ainda dá para fazer rollback.
`s.commit()`	Finaliza a transação e **torna as mudanças permanentes**.

```
pedido = Pedido()
s.add(pedido)
s.flush()           # pedido.id já existe aqui
item = ItemPedido(pedido_id=pedido.id, ...)  # usa o id gerad
o
s.add(item)
s.commit()          # persiste pedido + item juntos
```

Resumo Visual

```
main.py
  |
  ├── from Controllers import ...
  |     ├── abre session
  |     ├── opera no Model (query, add, delete)
  |     ├── commit / rollback
  |     └── fecha session, retorna objetos
  |
  ├── from Views import ...
  |     └── recebe objetos → print formatado
  |
  └── coordena: chama controller → passa resultado para view
```